PART 3 - MODULES AND FUNCTIONS



- INTRODUCTION
- THE NUMPY MODULE
- PRACTICAL 3



all work and no play makes Dave a dull boy...

David Topping: david.topping@manchester.ac.uk

'Vanilla' Python

Data types
Basic operations

Loops
Conditionals

Pandas and data discovery

Geospatial analysis

Statistical Analysis

Cluster Analysis

1 2 3 4 5 6 7 8 9

The Numpy module

Conway's Game Of Life

Differential Equations

Modules and Functions

Creating a notebook assessment
select topic from weeks 4 - 9

10 11 12

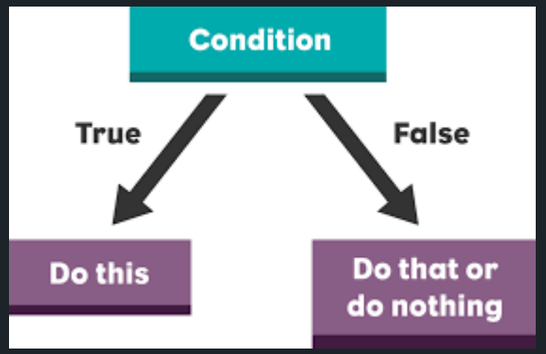
COURSE LAYOUT >>>>

LECTURE + PRACTICAL. SOLUTIONS ARE GIVEN

RECAP

- Loops and conditionals

we have a workflow



we have operators

Meaning	Math Symbol	Python Symbols
Less than	<	<
Greater than	>	>
Less than or equal	≤	<=
Greater than or equal	≥	>=
Equals	=	==
Not equal	≠	!=

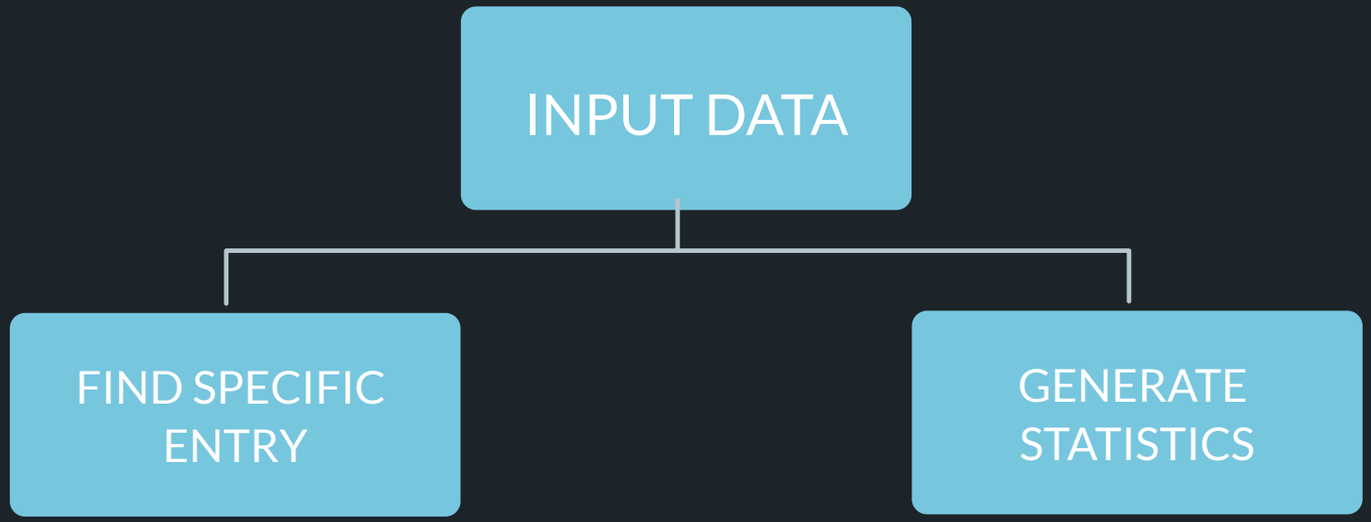
just because it works does not make it correct

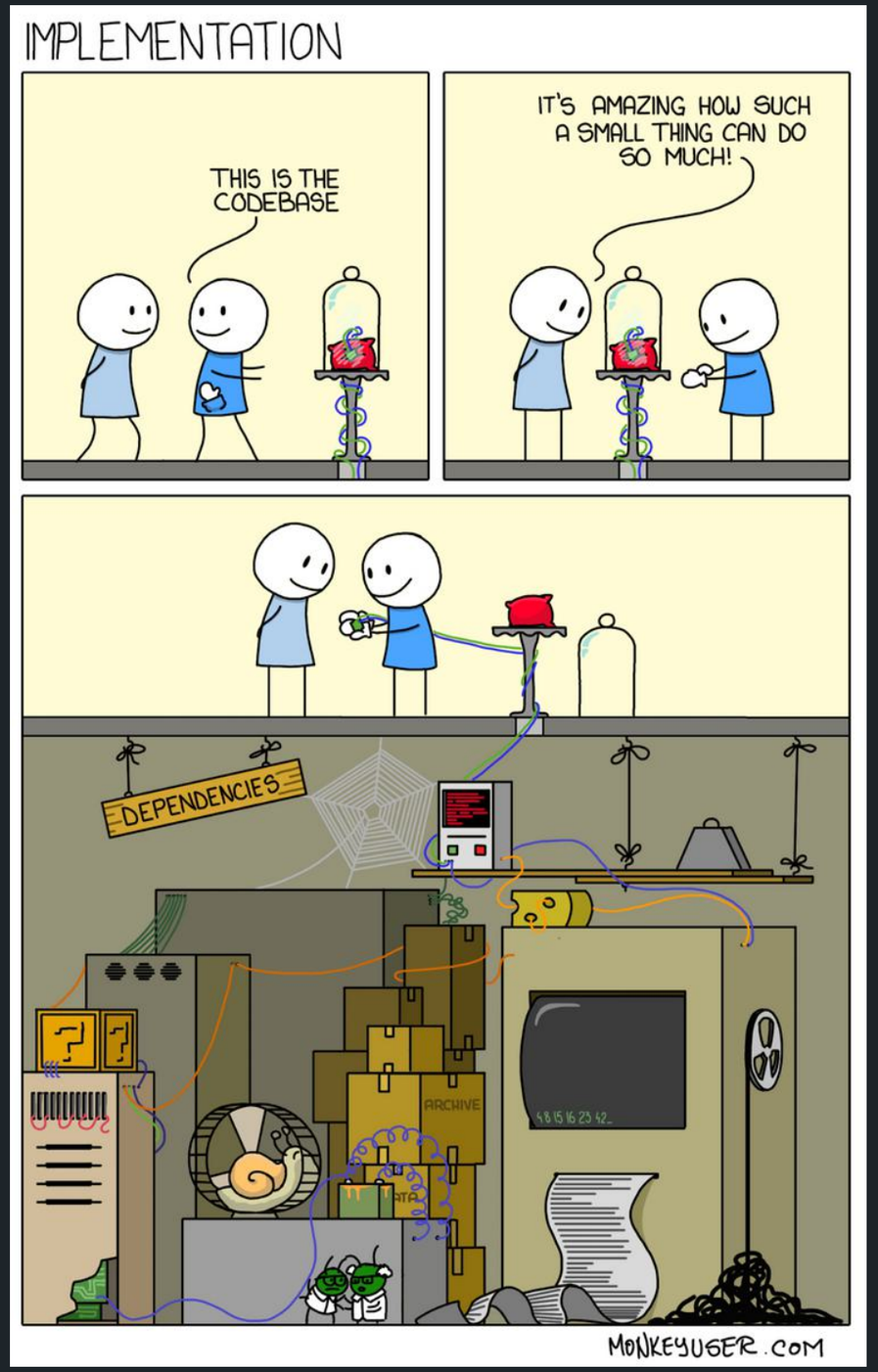
```
# Version 1
if x[step] < 20:
    <<action>>
elif x[step] >= 20:
    <<action>>
elif x[step] >= 60
    <<action>>

# Version 2
if x[step] < 20:
    <<action>>
elif x[step] >= 20 and x[step] < 60:
    <<action>>
elif x[step] >= 60
    <<action>>
```

CONSTRUCTING A MODULAR PROGRAM HAS MULTIPLE BENEFITS:

- Sharing
- Independent testing
- Version control...





FUNCTIONS

e.g. printing to a screen

In [ ]:
```
# Lets define a function called 'test'. This function will simply write a message to the screen.
def test():
    print("All work and no play makes Jack a dull boy")
    return

# Call the function
test()
```

SYNTAX CHECK!

- The 'def' keyword defined a function by its name.
- The parenthesis () can take a list of inputs that the function needs.
- The function name and input list is terminated with a colon ':'
- The next line, thus internal workings of the function, is idented by 4 spaces.
- A 'return' statement closes the function definition.

FUNCTIONS

e.g. printing to a screen

In [ ]:
```
# Lets define a function called 'test'. This function will simply write a message to the screen.
def test():
    print("All work and no play makes Jack a dull boy")
    return

# Call the function
test()
```

e.g. How do we call this function 'test' 30 times?



MODULES CONTAIN FUNCTIONS AND VARIABLES THAT CAN BE ACCESSED BY IMPORTING THAT MODULE FROM OUR 'MAIN SCRIPT'

We import the module that has the same name as a file containing all functionality

```
>> import module_name
>> module_name.greeting("David")
>> Hello, David
```

module_name.py

```
# Function to say hello
def greeting(name):
  print("Hello, " + name)
  return
```

The Python interpreter has to 'see' the module so typically this is either in the same directory, or in our 'PYTHONPATH' or we can point the program to the location of the module

MODULES CONTAIN FUNCTIONS AND VARIABLES THAT CAN BE ACCESSED BY IMPORTING THAT MODULE FROM OUR 'MAIN SCRIPT'

We import the module that has the same name as a file containing all functionality

```
>> import module_name
```

module_name.py

```
Import module1
Import module2 as MD2

# Function to say hello
def greeting(name):
  print("Hello, " + name)
  return
```

module1.py

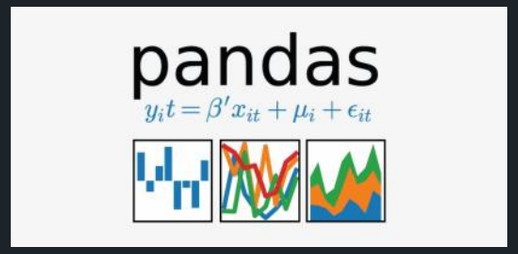
module2.py

```
>> module_name.greeting("David")
>> Hello, David
```

The Python interpreter has to 'see' the module so typically this is either in the same directory, or in our 'PYTHONPATH' or we can point the program to the location of the module

WE ARE GOING TO USE A NUMBER OF MODULES









In [ ]:
```
import numpy as np
import pandas as pd
import sklearn #the name of the module is different from the official title, but the .py will have this name
import matplotlib
```

Sometimes we dont want to import an entire module if we are only using one function out of many. As an example, in the Scikit-learn package I might only want to import a function that allows me to perform a technique known as K-means clustering. By checking the online documentation, I find that I should import this as:

In [ ]:
```
from sklearn.cluster import KMeans
```

Does the syntax make sense? The use of '.' is important. This is allowing me to access functions within the entire package. We will use this today as we explore the functionality of the Numpy package.

PART 3 - MODULES AND FUNCTIONS



- INTRODUCTION
- THE NUMPY MODULE
- PRACTICAL 3



all work and no play makes Dave a dull boy...

David Topping: david.topping@manchester.ac.uk



WE ARE GOING TO STOP USING LISTS FROM THIS POINT ON....

In [2]:
```
# Lets import the numpy package and call it np
import numpy as np

# We can use an internal Numpy function to create a 1D array of fixed length. Lets say we want to create arrays
# with 15 elements. For this Im going to use the function numpy.zeros() to create an array filled with 0's.
# The official documentation for this function can be found here: https://docs.scipy.org/doc/numpy/reference/generated/
# Lets look at our example for now

# Im going to use my own variable names for each array. This dosnt matter:
Integer_array = np.zeros((15,), dtype=int)
Float_array = np.zeros((15,), dtype=float)
```

Before we print these arrays to the screen, lets see what has happened here. The function **numpy.zeros()** is expecting at least two bits of information

> numpy.zeros((rows,columns), type of data)

Where, in our case, we have specified we want 15 entries, but only in one dimension as we have ignored the 'columns' variable. If we wanted to specify a 2D array, or matrix, as we will do later with 15 rows and 4 columns, we would write:

> Integer_array = np.zeros((15,4), dtype=int)

Notice we also specified our data type in each case. The first uses **'dtype=int'** and the second **'dtype=float'**. The function **expects** this syntax, as you might



We can access entries, and modify values using individual cells or straddle rows, columns

In [4]:
```
# Create an empty matrix, of 3 rows and 3 columns, using the numpy.zeros() function
matrix = np.zeros((3,3), dtype=float)

# Now lets put the value 298.15 in each cell. There are multiple ways we could do this. I demonstrate 4
# below, where one method is significantly quicker than the other from a data entry perspective

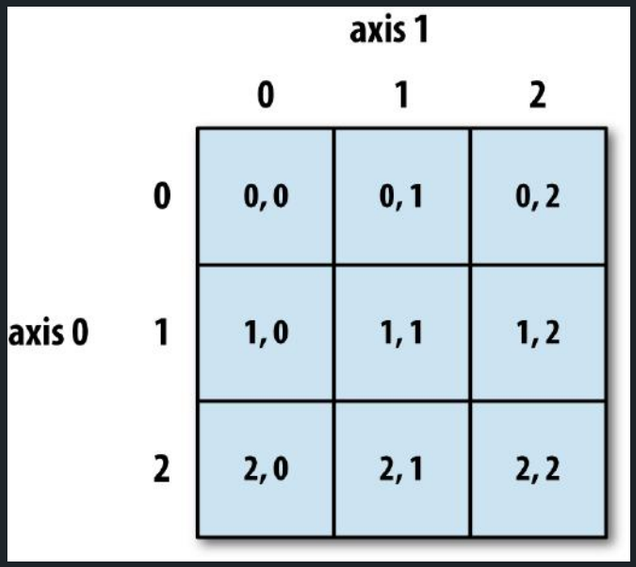
# 1) Enter all values manually. 1st row and 1st column, 1st row and 2nd column etc etc
matrix[0,0] = 298.15; matrix[0,1] = 298.15; matrix[0,2] = 298.15
matrix[1,0] = 298.15; matrix[1,1] = 298.15; matrix[1,2] = 298.15
matrix[2,0] = 298.15; matrix[2,1] = 298.15; matrix[2,2] = 298.15
# Lets print the matrix to the screen
print(" first entry method result = ", matrix)

# 2) Assign the value to every column as a function of each row using the ':' operator
matrix = np.zeros((3,3), dtype=float)
matrix[0,0:3] = 298.15
matrix[1,0:3] = 298.15
matrix[2,0:3] = 298.15
# Lets print the matrix to the screen
print(" second entry method result = ", matrix)

# 3) Assign the value to every row as a function of each column using the ':' operator
matrix = np.zeros((3,3), dtype=float)
matrix[0:3,0] = 298.15
matrix[0:3,1] = 298.15
matrix[0:3,2] = 298.15
# Lets print the matrix to the screen
print(" third entry method result = ", matrix)

# 4) Assign every cell using the ':' operator in 2 dimensions
matrix = np.zeros((3,3), dtype=float)
matrix[0:3,0:3] = 298.15
# Lets print the matrix to the screen
print(" fourth entry method result = ", matrix)
```

We access entries using the same indexing as before



The code to the left shows 4 ways of performing the same operation

E.G. CREATING A 2D SURFACE OF Z VALUES BASED ON AN X AND Y AXIS

What about accessing and modifying values in a 2D array? We can 'nest' multiple *for* loops to deal with each dimension. Below is an example where we populate values in a 2D array, of 100 rows and 100 columns, according to the following function:

$$z = x + y$$

Where variable 'X' represents a row index and 'Y' represents the column index. 'Z' is a 2D array which is then plotted using the matplotlib package, which is given assuming you use a 2D Numpy array for Z. Read through the comments associated with each step, can you recognise the stages of constructing this set of operations?

In [2]:
```
# Import the modules we will be using
import numpy as np
import matplotlib.pyplot as plt

# Initialise the Numpy arrays
x = np.zeros((100,), dtype=float)
y = np.zeros((100,), dtype=float)
z = np.zeros((100,100), dtype=float)

# Lets populate our x and y arrays with values from 1 to 100
for step in range(100):
    x[step]=step+1 # why do we add 1?
    y[step]=step+1 # why do we add 1?

# Now create a nested loop that cycles through each dimension to produce our z array
# Thus the first loop cycles through each row
for row in range(100):
    # The second loop cycles through each column.
    for column in range(100):
        z[row,column]=x[row]+y[column]
```

e.g. Writing a function to perform the numerical operation z=x+y*x

without function

```
# Import the modules we will be using
import numpy as np
import matplotlib.pyplot as plt

# Initialise the arrays
x = np.zeros((100,), dtype=float)
y = np.zeros((100,), dtype=float)
z = np.zeros((100,100), dtype=float)

#  ------- 'INSERT CODE HERE' ---------
# Lets populate our x and y arrays with values from 1 to 100
for step in range(100):
    x[step]=step+1 # why do we add 1?
    y[step]=step+1 # why do we add 1?

#  ------- 'INSERT CODE HERE' ---------
# Now create a nested loop that cycles through each
dimension to produce our z array
# Thus the first loop cycles through each row
for row in range(100):
    # The second loop cycles through each column.
    for column in range(100):
        z[row,column]=x[row]+y[column]*x[row]

# Plot the results. Dont worry about the Matplotlib
template for now.
plt.contourf(x, y, z, 20, cmap='RdGy')
plt.colorbar();
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

with function

```
# Import the modules we will be using
import numpy as np
import matplotlib.pyplot as plt

# Initialise the arrays
x = np.zeros((100,), dtype=float)
y = np.zeros((100,), dtype=float)

# Lets populate our x and y arrays with values from 1 to 100
for step in range(100):
    x[step]=step+1 # why do we add 1?
    y[step]=step+1 # why do we add 1?

# Implement the equation z=x+y as a function
def create_z_array(x,y):
    # Initialise the z array
    z = np.zeros((100,100), dtype=float)
    # Now create a nested loop that cycles through
    # each dimension to produce our z array
    # Thus the first loop cycles through each row
    for row in range(100):
        # The second loop cycles through each column.
        for column in range(100):
            z[row,column]=x[row]+y[column]

    # return the z array
    return z

# Call the function that generates our z array
z=create_z_array(x,y)

# Plot the results. Dont worry about the Matplotlib template for now.
plt.contourf(x, y, z, 20, cmap='RdGy')
plt.colorbar();
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

OPERATIONS ON ARRAYS RATHER THAN CELL-BY-CELL: 1) ARRAY ADDITION

A nice example is provided by using arithmetic operators with Numpy. If we have 2 arrays, array1 and array2, created through:

```
array1 = np.array([10,20,30,40,50], dtype=int)
array2 = np.array([0,1,2,3,4], dtype=int)
array3 = array1+array2
```

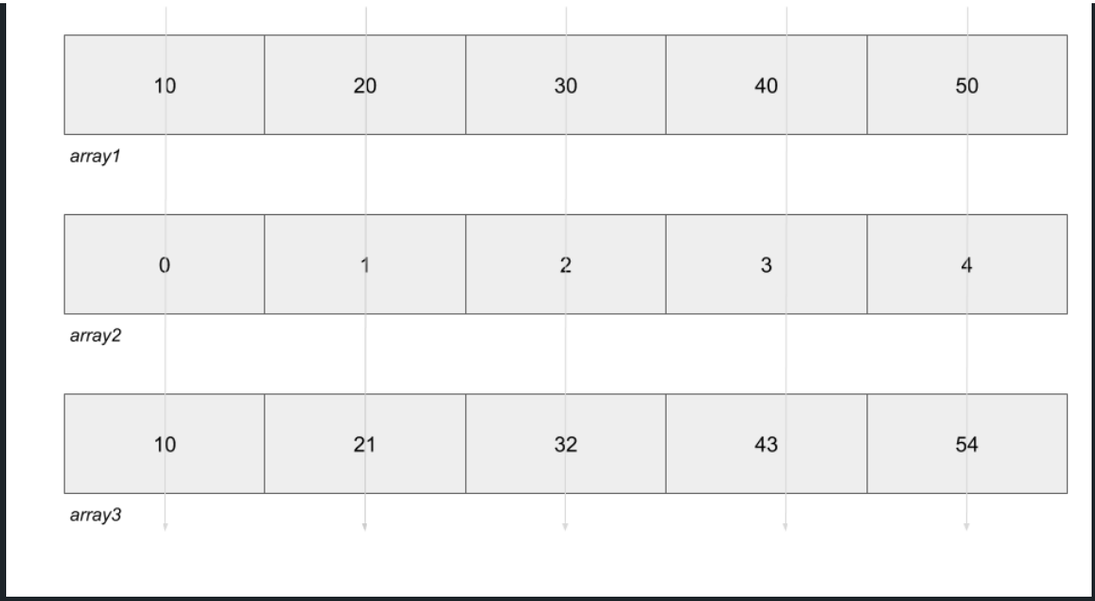
what might our answer look like? Lets run this code below and print to the screen

In [1]:
```
import numpy as np

array1 = np.array([10,20,30,40,50], dtype=int)
array2 = np.array([0,1,2,3,4], dtype=int)
array3 = array1+array2

print("array3 = ", array3)
```

```
array3 =  [10 21 32 43 54]
```



OPERATIONS ON ARRAYS RATHER THAN CELL-BY-CELL: 1) ARRAY MULTIPLICATION

In [1]:
```
import numpy as np

array1 = np.array([10,20,30,40,50], dtype=int)
array2 = np.array([0,1,2,3,4], dtype=int)
array3 = array1*array2

print("array3 = ", array3)
```

```
array3 =  [  0  20  60 120 200]
```

In [2]:
```
import numpy as np

array1 = np.array([10,20,30,40,50], dtype=int)
array2 = np.array([0,1,2,3,4], dtype=int)
array3 = np.multiply(array1,array2)

print("array3 = ", array3)
```

```
array3 =  [  0  20  60 120 200]
```

OPERATIONS ON ARRAYS RATHER THAN CELL-BY-CELL: 1) ARRAY MULTIPLICATION

In [4]:
```
# Initiliase two empty matrices, A and B
A = np.zeros((3,3), dtype=float)
B = np.zeros((3,3), dtype=float)

# Set all cells in A to 2.4 and all cells in B to 1.6.
A[:,:]=2.5
B[:,:]=2.0

# Lets print the result of adding both together
print(A*B)
```

```
[[5. 5. 5.]
 [5. 5. 5.]
 [5. 5. 5.]]
```

Elementwise multiplication is not the same as matrix multiplication

PART 3 - MODULES AND FUNCTIONS



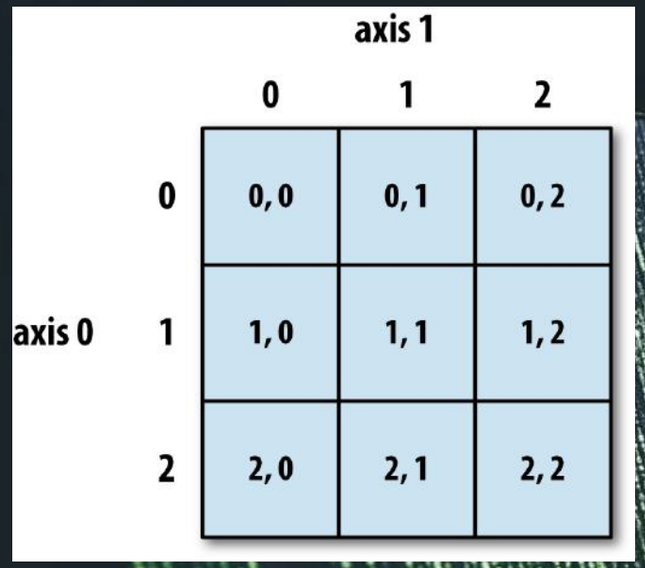
- INTRODUCTION
- THE NUMPY MODULE
- PRACTICAL 3



all work and no play makes Dave a dull boy...

David Topping: david.topping@manchester.ac.uk

ARRAY SLICING



Using the : operator for array slicing. What happens behind the scenes?

a:b -> Python interpreter -> a, a+1, a+2,.... b-1
a:n:b -> Python interpreter -> a, a+n, a+2n,.... b-n

1) Setting a collection of cells to one value

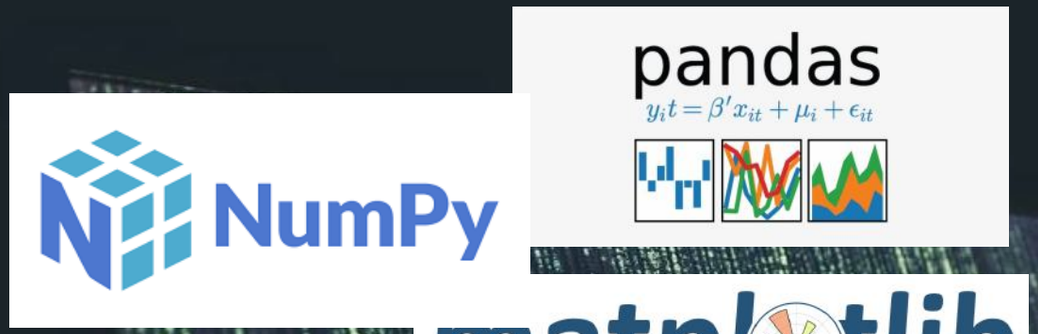
Python searches for the location of each cell in the iteration and returns 'null' if there isn't one, but doesn't throw an error

e.g.
array[0,0:2]=3.0 works
array[0,0:20]=3.0 works

2) Mapping a collection of cells to another collection of cells.

They have to match perfectly.....

LECTURE 3






```
In [1]: import numpy as np

        array1 = np.array([10,20,30,40,50], dtype=int)
        array2 = np.array([0,1,2,3,4], dtype=int)
        array3 = array1*array2

        print("array3 = ", array3)

        array3 =  [  0  20  60 120 200]
```

SUMMARY

MODULES AND FUNCTIONS

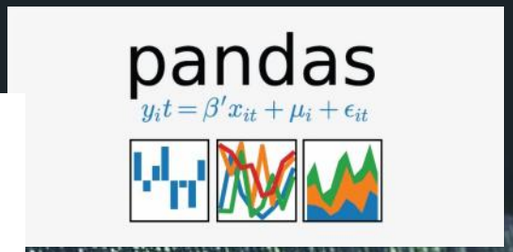
- Allow us to build programs in a modular way
- Improve sharing and error mitigation
- Allow us to use mature community modules/packages for a particular purpose

THE NUMPY MODULE

- A more efficient way of dealing with numerical arrays
- Similar to Vanilla Python
- Used as standard in Python programs

PRACTICAL 3








```
In [1]: import numpy as np

        array1 = np.array([10,20,30,40,50], dtype=int)
        array2 = np.array([0,1,2,3,4], dtype=int)
        array3 = array1*array2

        print("array3 = ", array3)

        array3 =  [  0  20  60 120 200]
```

GOALS

CODE

- Learn how to use the Numpy module
- Use conditional statements to modify values in an array using Numpy
- Create your own function
- Perform Numpy operations on entire arrays

GENERAL EXPERIENCE

- Start thinking in terms of functions
- Start thinking about how you might need to share your code!